

Тема 12 ВИДИМОСТ, РЕАЛИСТИЧЕН ЦВЯТ

Отстраняване на невидими линии и повърхнини. Модели на осветяване. Генерация на реалистични изображения. Системи с тримерни изображения. OpenGL – Нормали и Динамична светлина

упр. 13 Реализация на алгоритми за отделяне на видимите повърхности. Постигане на реалистичност на изображенията

12.1 ПРЕМАХВАНЕ НА СКРИТИ ЛИНИИ И ПОВЪРХНОСТИ

12.1.1 Въведение

Проблемът за видимостта на обекта върху изображението се решава като задача за отделяне на невидими линии и задача за отделяне на невидими повърхности. Разделянето на видими и невидими елементи на сцената може да се осъществи както в пространството на обектите, така и в пространството на изображението. В първия случай определянето на това, което се вижда става на основата на геометрични съотношения между частите на обекта. Във втория случай се осъществява обход на изображението и изучаване на проекцията по пътя на неговото построяване.

Нека да предположим, че е генериран в съответствие с изложеното в предишните глави тримерен обект, като са определени типа на проекцията, видовата равнина и др. Необходимо е да се определи кои ребра и повърхности на този обект са видими, ако се гледа от центъра на проекцията (или от някаква друга точка) при централната проекция или по посока на направлението на проектиране за паралелните проекции. В такъв случай, за създаване на максимална илюзия за тримерност, би следвало да се извеждат на екрана само онези ребра и повърхности, които се виждат. Тази задача се формулира просто, но за нейното решение е необходимо **голямо процесорно време и щателно подготвени и структурирани алгоритми**.

Определение: Стена S_i закрива (засенчва) или обгражда стената S_j по направление γ при изпълнение на следните условия:

- а) Стената S_i не пресича стената S_j ;
- б) Ортогоналните проекции на стените S_i и S_j в равнина перпендикулярна на направление γ имат някакво непразно сечение T_{ij} ;
- в) Най-малко една от точките на стена S_i , проекцията на която принадлежи на T_{ij} , се намира по-далеч от началото на координатната система по направлението γ отколкото съответната точка на S_j . Приема се че направлението γ съвпада с оста Z .

Съществуват **два подхода** за решението на този проблем.

При **първия** се разглеждат сечения на всички страни с някаква хоризонтална плоскост и за получените в резултат отрязъци от прави се решава задачата за разделяне на видими и невидими елементи от сцената. В случая обектът се разглежда като съвкупност от n -ъгълни стени и е необходимо да се определи, коя от тях се вижда във всяка разрешаваща точка на екрана. За да се покаже при това, коя от стените се намира по-близо до наблюдателя е необходимо да се проверят всичките n стени за всяка разрешаваща точка! За N такива точки броят на проверките е пропорционално на $n \cdot N$, където N обикновено се намира в диапазона от 250 000 до 4 000 000.

При **втория** подход се разглеждат многоъгълници, явяващи се проекции на всяка от n -те стени и с алгоритъм се търси пресичането с останалите $n-1$ стени. Броят на проверките в този случай е пропорционален на n^2 . Затова може да се предположи, че този подход е по-добър даже при най-големите и реално срещани стойности на n - от порядъка на 100 000 до 200 000. Обаче, всяка от индивидуалните стъпки при този подход изисква много време, и затова този процес ще се окаже по-бавен, даже при по-малки n .

Някои автори, като Шумахер (виж [2]), са назвали тези два подхода като алгоритми, работещи съответно в **пространството на изображението** и в **пространството на обекта**.

Броят на алгоритмите за премахване на скритите линии е голям - над 15. По-долу ще бъдат разгледани само два. Други могат да се видят в [11].

Алгоритмите за премахване на скритите повърхности е необходимо да се построяват така, щото всяка тяхна стъпка да бъде колкото може по-ефективна. Съществуват твърде много алгоритми за премахване на скрити повърхности върху изображенията на тримерни обекти, съставено от полигонни стени. Някои от тези алгоритми могат да се използват за

премахване и на скритите ребра. Читателят може да види някои от тези идеи в [11]. Тук ще бъдат накратко изложени алгоритми за изобразяване на обекти, зададени с парчета криволинейни повърхности. Тези случаи се оказват едни от най-сложните и ресурсоемки.

12.1.2. Алгоритъм на плаващия хоризонт за премахване на скрити линии

Тук тримерната задача се свежда до двумерна чрез пресичане на изходната повърхност с последователност от паралелни секущи равнини. Пресечниците се проецират на равнината $z=0$. Нека за пример имаме функцията $F(x,y,z)=0$. Тогава координатите на всяка точка могат да се запишат така $y=f(x,z)$ или $x=g(y,z)$ където $z=\text{const}$. Ако получените криви се проектират на равнината $z=0$, то става ясна идеята за алгоритъм за премахване на скритите линии. Първо, подреждат се (сортират се) равнините $z=\text{const}$ по нарастване на разстоянията до тях от точката на виждане (наблюдение) и второ, на всяка равнина, започвайки от най-близката се строи крива, лежаща на нея. В такъв случай Алгоритъмът за премахване на скритите линии се състои в следното:

Ако на текущата равнина, при някакво зададено x съответното y на кривата е по-голямо от y за всички предни криви при тази стойност на x , то текущата крива е видима в тази точка. В противен случай е невидима.

Алгоритъмът работи твърде добре до тогава, докато някоя поредна крива не се окаже по-долу от най-първата. В такъв случай следва да се използва следната модификация в предложения алгоритъм:

Ако на текущата равнина при зададено x съответното y на кривата е по-голямо от максимума или е по-малко от минимума по y за всички предидущи криви при това x , то текущата крива е видима в тази точка. В противен случай е невидима.

Предлага се на читателя да състави програма, реализираща предложения алгоритъм за произволна функция $F(x,y,z)$, зададена от него самия.

12.1.3. Алгоритъм, използващ Z-буфер

Това е най-простия алгоритъм за премахване на скритите линии. Работи в пространството на изображението. Предложен е от Кетмул [11]. Тази идея е обобщение на идеята за **буфер на кадъра** [4]. Той пък се използва за запомняне на атрибутите (интензивностите) на всеки пиксел в пространството на изображението. Z-буферът е отделен буфер на **дълбочината**, използван за запомняне на координата z или на всеки **видим пиксел** в пространството на изображението. В процеса на работата дълбочината или стойността z на всеки нов пиксел се сравнява с дълбочината на същия пиксел, който вече е внесен в Z-буфера. Ако новият е пред стария, то той се внася в буфера и се прави корекция на Z-буфера с новата стойност на z . Иначе всичко останало остава по старому. Или за даден пиксел с координати (x,y) се търси най-голямата стойност на $z(x,y)$.

Простотата на този алгоритъм е главното му предимство. Големият обем памет, необходим за реализацията му, е главният му недостатък. Но с поевтиняването на последната е икономически оправдано създаването на специализирани запомнящи устройства за Z-буфер и свързаната с него апаратура във всеки мощен в графично отношение компютър. Например, нека за информацията по дълбочина са нужни 20 бита. Тогава буфер на кадъра с размер $512 \times 512 \times 24$ бита в комбинация със Z-буфер с размер $512 \times 512 \times 20$ бита изисква почти 1.5 Мбайта памет.

12.1.4. Алгоритъм за полинейно сканиране, използващ Z-буфер

Това е един прост алгоритъм за полинейно сканиране, който решава задачата за премахване на **невидимите повърхности** и представлява специален случай на алгоритъма на Z-буфера. При него се прилага така наречения **прозорец на визуализацията**, имащ височина една сканираща линия и ширина, равна на ширината на целия екран. Както за буфера на кадъра, така и за Z-буфера е необходима памет с височина 1 бит, ширина равна на хоризонталния размер на екрана и дълбочина в зависимост от нужната точност. Тя зависи от своя страна от диапазона на стойностите, които може да заема z . Например буферът на кадъра може да има размер $1 \times 512 \times 24$ бита, а Z-буфера - $1 \times 512 \times 20$ бита.

Концептуално този алгоритъм просто предполага, че за всяка сканираща линия буферът на кадъра се инициализира с фоновата стойност на интензивност, а Z-буфера - с минималната стойност на z. След това се определят пресичанията на сканиращата линия с двумерната проекция на всеки многоъгълник от сцената, ако те съществуват. Тези пресичания образуват двойки, които бяха разглеждани при алгоритмите за сканиране на запълнени области. При разглеждането на всеки пиксел на сканиращата линия в интервала между краищата на двойките неговата дълбочина се сравнява с дълбочината, съдържаща се в съответния елемент на Z-буфера. Ако дълбочината на този пиксел е по-голяма от дълбочината, взета от Z-буфера, то разглежданият отрязък от линията (сканиращата) ще бъде видим. И следователно атрибутите на многоъгълника, съответстващи на дадения отрязък се внасят в буфера на кадъра в позицията на дадения пиксел, коригира се и Z-буфера в тази позиция. След обработката на всички многоъгълници от сцената буферът на кадъра с размер на една сканираща линия съдържа решението на задачата за премахване на невидимите повърхности за дадената сканираща линия.

12.1.5. Алгоритъм за отстраняване на невидими части от криволинейни области

Разгледаните по-горе алгоритми са приложими само за обекти, състоящи се от плоски стени. За да се използва който и да е от тях за изобразяване на обекти, ограничени от криволинейни повърхности е необходимо предварително да се апроксимират тези обекти с голямо количество малки плоски стени. Това може да се направи, но естествено значително по-удобно би било да се работи непосредствено с криволинейните повърхности, например тази на Безие, която бе разгледана в предните глави.

В теорията и практиката на компютърната графика са известни множество алгоритми за решението на този клас задачи, но различните от тях имат свои решения. Например съществуват алгоритми за отстраняване на скрити повърхности от квадратичен вид - сфера (една повърхност), затворен цилиндър (три повърхности), затворен конус (две повърхности) и елипсоид (една повърхност). В общия случай за произволна повърхност тази задача се решава, но достатъчно трудоемко и ресурсоемко. Редица автори предлагат такива алгоритми, като за квадратичните повърхности те се базират на пресичането им, определянето на което води до уравнения от четвърти ред относно x, y и z . Техните корени се намират чрез подходящи числени методи. Сферите обаче, явяващи се частен случай на квадратичните повърхности, представляват особен интерес, защото компютърното моделиране в редица случаи, например, в биологията, където молекулите се изобразяват във вид на съвкупност от оцветени в различен цвят сфери, би изисквало качествени и бързи алгоритми за откриване на невидимите части от тези цели комплекси от сфери.

По универсални се явяват параметричните бикубични повърхности, разгледани в Глава 18. Те имат по-голяма общност и осигуряват непрекъснатост на производните на границата на отделните късове, изграждащи един по-сложен обект. Кетмул е предложил първи алгоритъм, позволяващ да се изобразяват такива повърхности. Той се основава на идеята за рекурсивно разделяне на къса по s и t до тогава, до като *поредният елемент*, получен чрез това разделяне не получи *проекционно изображение на екрана примерно равно на един пиксел*. Определянето на *видимостта на такава малка област* относно всички преди това разгледани аналогично късове, се *изпълнява с помощта на алгоритми използващи Z-буфер*. След това се изчислява оцветяването на тази част от повърхността (която е видима) съгласно известните модели и съответни алгоритми, показани в Глава 19 и се внася в буфера на кадъра.

Съществуват алгоритми, работещи непосредствено с параметричното представяне на повърхностите. Такъв е алгоритъмът на Блин [11]. За сканиращата линия $y=a$ се определят стойностите на s и t , удовлетворяващи уравнението

$$y(s,t) - a = 0. \quad (12.1)$$

След това тези стойности се използват за изчисляване на $x(s,t)$ и $z(s,t)$. За съжаление обаче, уравнението (12.1) не се решава в общ случай и затова корените му се следва да се определят числено по някои от известните числени методи, например този на Нютон. Вижда

се огромната трудоемкост при решението на тази задача. Когато уравнението няма корени този алгоритъм не работи.

В заключение може да се каже, че *най-ефективни* понастоящем са алгоритмите, основани на *подхода за адаптивно разделяне* на бикубичните късове до тогава, до като поредните парченца не станат *достатъчно плоски*. След това малките многоъгълници, зададени с четирите си ъглови точки на всеки от получените в резултат на делението късове, се обработват с помощта на алгоритъма за полинейното сканиране, разгледан по-горе. Така се получава интересна възможност за едновременно използване на многостенни и бикубични повърхности, което до голяма степен решава задачата за генериране на сложни реалистични тримерни изображения.

Следва особено да отбележим съществуването на ефективни алгоритми за решаване на разглежданата в настоящата глава задача въз основа на използването на метода на цифровата интегрираща машина. Те обаче няма да бъдат разгледани в настоящата книга, а ще бъдат изложени в специално подготвена от авторите монография.

12.2. СЪЗДАВАНЕ НА РЕАЛИСТИЧНИ ИЗОБРАЖЕНИЯ

Реалистичното изображение се постига с отделяне на скритите повърхности в зависимост от гледната точка и оцветяване на видимите повърхности. Външният вид на повърхността зависи от вида на източника на светлина и от свойствата на повърхността относно източника на светлината. Повишаване на реалността се постига с възпроизвеждане на сенките, които хвърля обекта. Построяването на реалистичното изображение включва както физически, така и психологически процеси.

Пътища за създаване на реалистични изображения.

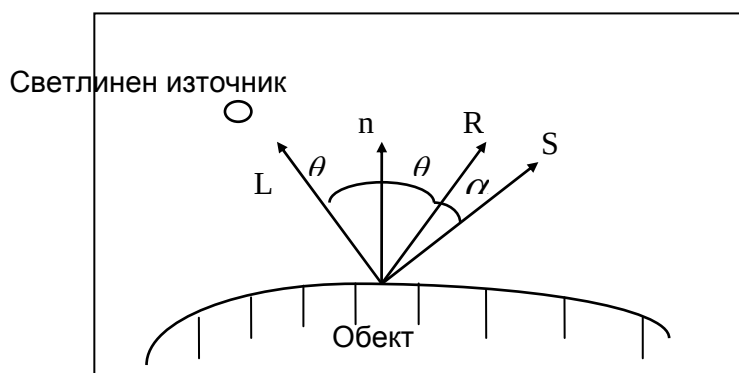
1. Отделяне на скритите повърхности.

2. Оцветяване на видимите повърхности.

Външният вид на повърхността зависи от вида на източника на светлина, който осветява обекта, от свойствата на повърхността (цвят, фактура, отражателна способност), а така също от положението и ориентацията на повърхността относно източника на светлина и другите повърхности.

12.2.1. Прост модел на осветяване

Простият модел на осветяване е представен на фиг. 12.2.1.



Фиг. 12.2.1

Светлинната енергия, падаща на повърхността, може да бъде погълната, отразена или пропусната. Частично тя се поглъща и се превръща в топлина, частично се отразява или пропуска. Обектът се вижда, ако той отразява или пропуска светлина. Ако той поглъща цялата светлина, той е невидим и се нарича абсолютно черно тяло. Количеството погълната, отразена или пропусната светлина зависи от дължината на вълните. При осветяване с бяла светлина, при която интензивността на всички вълни е снижена еднакво, обектът изглежда сив. Ако се поглъща почти цялата светлина, обектът изглежда черен, а ако се поглъща съвсем малко светлина - бял. Ако се поглъщат само определени дължини на

вълните, то светлината отразяваща се от обекта се изменя и обектът изглежда цветен. Цветът се определя от погълнатите вълни с определена дължина.

Свойствата на отразената светлина зависи от строежа и формата на източника на светлина и от ориентацията и свойствата на повърхността. **Отразената** от обекта светлина може да бъде **дифузна** и **огледална**. Дифузно е отражението на светлината, когато тя като че ли прониква под повърхността на обекта, поглъща се и отново се изпуска. В този случай наблюдателят не е от значение, т.к. светлината се разсейва равномерно по всички направления. Огледалното отражение става от външната повърхност на обекта.

12.2.1.1. Дифузно отражение

Светлина от светлинен източник се отразява от идеалния разсейвател по косинусовия закон на Ламберт

$$I_d = I_l \cdot k_d \cdot \cos \theta, \quad 0 \leq \theta \leq \pi / 2 \quad (12.2.1)$$

където:

I_d - интензивност на отразената светлина;

I_l - интензивност на точковия източник;

k_d - коефициент на дифузно отражение от 0 до 1. Коефициентът k_d е различен за трите основни цвята -син k_{ds} , червен k_{dr} и жълт k_{dy} , като определящ е цветът на повърхността.

Например при жълта повърхност $k_{ds} = 0, k_{dr} = 0, k_{dy} = 1$.

θ - ъгъл между направлението на светлината и нормалата към повърхността.

Ако $\theta > \pi / 2$, то източникът се намира зад обекта. Коефициентът k_d зависи от материала и дължината на вълните на светлината, но в простите варианти на модели от типа (12.2.1) се приема за постоянно.

Едновременно върху обекта от реалната сцена пада и **разсеяна** светлина, разсеяна от окръжаващата среда. На **разсеяната** светлина се приема да съответства **разпределен** източник за да се спестят големи изчислителни загуби. Разчетите за източниците за разсейване се заменят с коефициенти на разсейване, влизащи във формулата на Ламберт:

$$I = I_a \cdot k_a + I_l \cdot k_d \cdot \cos \theta, \quad 0 \leq \theta \leq \pi / 2 \quad (12.2.2)$$

където:

I_a - интензивност на разсеяната светлина;

k_a - коефициент на дифузно отражение на разсеяната светлина от 0 до 1. Коефициентът

k_a в (12.2.2) е различен за трите основни цвята -син k_{as} , червен k_{ar} и жълт k_{ay} , като определящ е цвета на източника на светлина.

Ако има два обекта, еднакво ориентирани към източника, но на различно разстояние, то по тази формула следва, че те еднакво ще се осветяват. Ако източникът на светлина е в безкрайността, то дифузния модел ще се превърне в 0, т.к. интензивността е обратно пропорционална на разстоянието до източника на квадрат. В случай на перспективно преобразуване на сцената в качеството на коефициент на пропорционалност на дифузния член може да се вземе d - разстоянието от центъра на проекция до обекта. Но ако центърът на проекция е много близо до обекта, то $1/d^2$ се изменя много бързо, т.е. за обекти лежащи на еднакво разстояние от източника, разликата в интензитета е много голяма. От опит се вижда, че при линейно затихване може да се постигне повече реалистичност. В този случай моделът на осветяване се изчислява по следния начин:

$$I = I_a \cdot k_a + \frac{I_l \cdot k_d \cdot \cos \theta}{d + k}, \quad (12.2.3)$$

където k е произволна константа.

Ако се предположи, че точката на наблюдение се намира в безкрайността, то d се определя от обекта най-близък до точката на наблюдение. Или най-близкият обект се

осветява с пълната интензивност на източника, а по-далечните с намалена. За цветните повърхности моделът (12.2.3) се прилага за всеки от основните цветове.

12.2.1.2. Огледално отражение

Интензивността на огледалното отражение зависи от ъгъла на падане, дължината на вълната на падащата светлина и свойствата на веществото на повърхността. Огледалното отражение се явява насочено. Ъгълът на отражение от идеална повърхност е равен на ъгъла на падане. Само ако векторът на наблюдението S съвпада с вектора на отражението R , наблюдателят вижда изображението. Във всяко друго положение наблюдателят не вижда отразената светлина. Ако повърхността не е идеална, то количеството светлина, достигащо до наблюдателя от пространственото разпределение на огледално отразената светлина е следното: При гладки повърхности е тясно и фокусирано, а при грапави - по-широко. Обикновено се използва следният модел:

$$I_s = I_l \cdot w(i, \lambda) \cdot \cos^n \alpha$$

където: $w(i, \lambda)$ - крива на отражение;

i - ъгъл на отражение;

λ - дължина на вълната;

n - степен по избор. Големите n дават фокусирани пространствени разпределения за блестящите повърхности (полирани твърди повърхности), а малките се използват за неподдаващи се на полиране повърхности, например хартия, кожа.

Функцията $w(i, \lambda)$ е достатъчно сложна, затова се заменя обикновено с коефициент k_s , който се избира от естетически съображения или се определя експериментално. За метали той е по-голям от 0.8, а при други материали под 0.04. Този коефициент е еднакъв за основните цветове.

Обединявайки моделите на дифузното и огледално отражение се получава модел, известен в компютърната графика като функция на оцветяването и се използва за пресмятане на интензивността, тона на точките от обекта или на всеки пиксел от изображението. За да се получи цветно изображение трябва да се намери функция на оцветяване за всеки от трите основни цвята. Константата k_s обикновено е еднаква за трите основни цвята. Тогава ще имаме следния модел на оцветяването:

$$I = I_a \cdot k_a + \frac{I_l}{d + k} (k_d \cdot \cos \theta + k_s \cdot \cos^n \alpha).$$

(12.2.4)

За m на брой източника на светлина функцията на оцветяването се пресмята като се обобщи (12.2.4) така:

$$I = I_a \cdot k_a + \sum_{j=1}^m \frac{I_{lj}}{d + k} (k_d \cdot \cos \theta_j + k_s \cdot \cos^n \alpha_j).$$

Ако запишем $\cos \theta = \frac{n \cdot L}{|n| \cdot |L|} = \bar{n} \cdot \bar{L}$, където $\bar{n}, \bar{L}$ са единични вектори на нормалата

към повърхността и направлението към източника на светлина и $\cos \alpha = \frac{R \cdot S}{|R| \cdot |S|} = \bar{R} \cdot \bar{S}$,

където $\bar{R}, \bar{S}$ са единични вектори, определящи направлението на отразения лъч и вектора на наблюдателя, тогава моделът на осветяване с един източник на светлина ще има вида:

$$I = I_a \cdot k_a + \frac{I_l}{d + k} [k_d \cdot (\bar{n} \cdot \bar{L}) + k_s \cdot (\bar{R} \cdot \bar{S})].$$

12.2.2. Методи на оцветяване

За да се получи цветно изображение трябва да се получат функциите на осветяване за всеки от трите основни цвята (син, червен и жълт). Константата k_d обикновено е еднаква за всичките три основни цвята, тъй като цветът на огледалното изображение се определя от падащия цвят. Цветът на разсеяната светлина има цвета на източника на светлина. За дифузното отразяване уравненията се записват отделно за синия, червения и жълтия цвят и

константите се задават с тройка числа (k_{ds}, k_{dr}, k_{dy}) . Когато падащият лъч е оцветен, то при отразяване неговите три допълнителни основни цвята ще имат интензивност, пропорционална на (k_{ds}, k_{dr}, k_{dy}) съответно, а разсеяната светлина ще има същия цвят като точковия източник. Трите уравнения имат вида:

- за синия цвят $I_s = I_{as} \cdot k_{as} + \frac{I_{ls}}{d+k} [k_{ds} \cdot (\bar{n} \cdot \bar{L}) + k_s \cdot (\bar{R} \cdot \bar{S})^n]$;

- за червения цвят $I_r = I_{ar} \cdot k_{ar} + \frac{I_{lr}}{d+k} [k_{dr} \cdot (\bar{n} \cdot \bar{L}) + k_s \cdot (\bar{R} \cdot \bar{S})^n]$;

- за жълтия цвят $I_y = I_{ay} \cdot k_{ay} + \frac{I_{ly}}{d+k} [k_{ds} \cdot (\bar{n} \cdot \bar{L}) + k_s \cdot (\bar{R} \cdot \bar{S})^n]$.

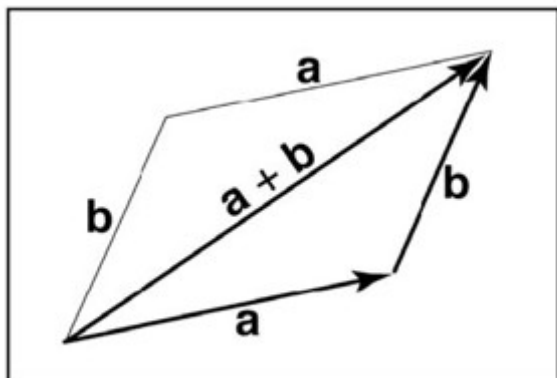
Повишаване на реалността може да се постигне по пътя на възпроизвеждането на сенките, които хвърля обектът, осветен от точков източник. Алгоритъмът за получаване на сенки в случай на точков източник е идентичен с този на скритите повърхности. В единия случай се отделят повърхности, които се виждат от наблюдателя, а в другия повърхности, които се виждат от точковия източник. Тези повърхности, които се виждат от източника и наблюдателя не са в сянка, а тези, които се виждат само от наблюдателя са в сянка. На повърхността, която е в сянка влияе само разсеяната светлина. По един от алгоритмите за отстраняване на скрити повърхности се разпознава кои повърхности се виждат от източника и от наблюдателя. На тази основа се осветява сцената.

Друг способ за достигане на реалистичност отчита свойствата на повърхността, като прозрачност и грапавина. Насоченото пропускане става през прозрачни повърхности. През тях добре се вижда, макар че лъчите се пречупват, т.е. отклоняват се от първоначалното направление. Вътрешни нееднородности в прозрачната повърхност водят до безпорядъчно смесване на светлинни лъчи и предмет под просветващото вещество има силно размити очертания. Детайлизацията на повърхността води до още по-добра реалистичност на изображението. Това става като се детайлизира грапавостта на повърхността.

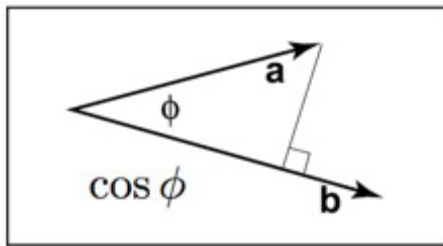
12.3 OPENGL - НОРМАЛНИ ВЕКТОРИ

Нормалите представляват вектори, които са перпендикулярни на лицето на даден полигон. Нормалните вектори са нужни при изчисленията на динамичната светлина. Ако се използват готовите обекти от помощните библиотеки (които са с определени нормални вектори), то определянето на нормалите за страните на обектите е задължително. За да се изчисли нормалния вектор е нужно да се вземе един от върховете на участъка (който може да бъде триъгълник, квадрат или друг), да се построят двата вектора към съседните два върха и да се умножат векторно.

Събиране и различни умножения на вектори.

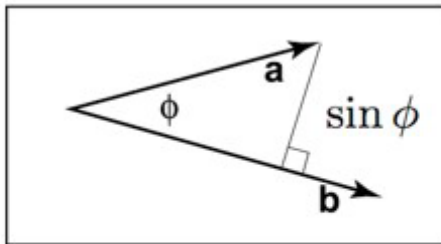


Скалярно умножение на вектори



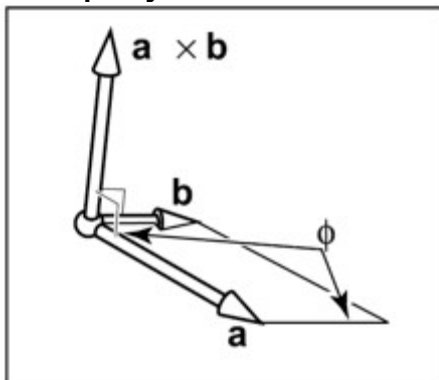
$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos \phi$$

Скалярно пресичащо се умножение на вектори



$$|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}| \sin \phi$$

Векторно умножение на вектори



$$\mathbf{c} = \mathbf{a} \times \mathbf{b}$$

$$\begin{aligned} x_c &= y_a z_b - z_a y_b \\ y_c &= z_a x_b - x_a z_b \\ z_c &= x_a y_b - y_a x_b \end{aligned}$$

Пример за определяне на нормали:

1. // дефиниране променливите в които се
2. // съхраняват x, y и z - позициите на три точки от фигурата
3. float Ax, Ay, Az;
4. float Bx, By, Bz;
5. float Cx, Cy, Cz;
6. // дефиниране два масива, в които ще се съхраняват векторите
7. // от A към B, от A към C и нормалния вектор към т.А
8. float Vab[3];
9. float Vac[3];
10. float N[3]
11. // намиране първия вектор AB
12. Vab[0] = Bx - Ax;
13. Vab[1] = By - Ay;
14. Vab [2]= Bz - Az;
15. // намиране втория вектор AC
16. Vac[0] = Cx - Ax;


```

17. Vac[1]= Cy - Ay;
18. Vac[2] = Cz - Az;
19. // намиране нормалния вектор
20. N[0] = Vab[1]*Vac[2] - Vab[2]*Vac[1]; //yazb-zayb
21. N[1] = Vab[2]*Vac[0] - Vab[0]*Vac[2]; //zaxb-xazb
22. N[2] = Vab[0]*Vac[1] - Vab[1]*Vac[0]; //xayb-zaxb
23. // нормализиране на нормалния вектор т.е.
24. // свеждане на неговата дължина по x, y и z в рамките на [0.0, 1.0]
25. float lenght;
26. lenght = 1.0 / sqrt( N[0]*N[0] + N[1]*N[1] + N[2]*N[2] );
27. N[0] *= lenght; N[1] *= lenght; N[2] *= lenght;

```

Така се изчислява нормалния вектор на дадена повърхност. Това се налага защото нормалата към участъка участва при изчисляването на реалистичен цвят. Там основната компонента е отразената светлина, която се определя от ъгъла на отразяване, който се измерва от мястото на светлинния източник и нормалата към повърхността на участъка.

За да се укаже на **OpenGL** текущия нормален вектор, за повърхността която предстои да бъде изрисувана, трябва да използвате функцията :

void glNormal3f(GLfloat nx, GLfloat, ny, GLfloat nz) - тя приема 3 аргумента и това са x, y и z координатите на нормалния вектор. Например :

```

1. glBegin( GL_TRIANGLES );
2. glNormal3f( N[0], N[1], N[2] );
3. glVertex3f( ... );
4. glVertex3f( ... );
5. glVertex3f( ... );
6. glEnd( );

```

За фигура, която има много страни (триъгълници или нещо друго) след определяне на нормалите на всяка страна) и пускане на програмата, то полигоните изграждащи фигурата си отразяват светлината, в зависимост от ъгъла под който тя пада върху тях. Всичко изглежда ниско-детайлно, изградено от отделни плочки и това е така защото наистина е изградено от отделни полигони и е ниско детайлно. За да се подобри изгледа може да се търси решение или като се направи модел с изключително много полигони или като се използват средни **върхови нормали**. Вторият подход е по-ефективен и за неговата реализация трябва да се изчислят по една нормала за всеки връх от **3D** обекта, вместо за всяка повърхност. Изчисляват се нормалните вектори за всички повърхности, които се намират до върха, събират се нормалните им вектори и се делят на броя им (на броя на повърхностите около върха) и се получи средната нормала за дадения връх. Благодарение на тази "технология" дори ниско-детайлните обекти изглеждат овални, все едно са изградени от хиляди полигони. Ако се разгледа триъгълна пирамида, то определянето на нормален вектор за един от четирите и върха се определя от 3-те нормали към повърхностите на страните на точката. Ако **N1[3], N2[3], N3[3]** са масивите в които са съхранени нормализираните нормални вектори на тези три страни, то нормалния вектор за върха между тях може да се изчисли по следния начин :

```

1. float Vbuffer[3]; // дефиниране масива, в който ще се съхранят сбора на векторите
2. float Vertex_Normal[3]; // нормалата за върха
3. // събиране на векторите
4. Vbuffer[0] = N1[0]+ N2[0] + N3[0];
5. Vbuffer[1] = N1[1]+ N2[1] + N3[1];
6. Vbuffer[2] = N1[2]+ N2[2] + N3[2];
7.
8. // изчисляване на нормалата за върха
9. Vertex_Normal[0] = Vbuffer[0] / 3;
10. Vertex_Normal[1] = Vbuffer[1] / 3;

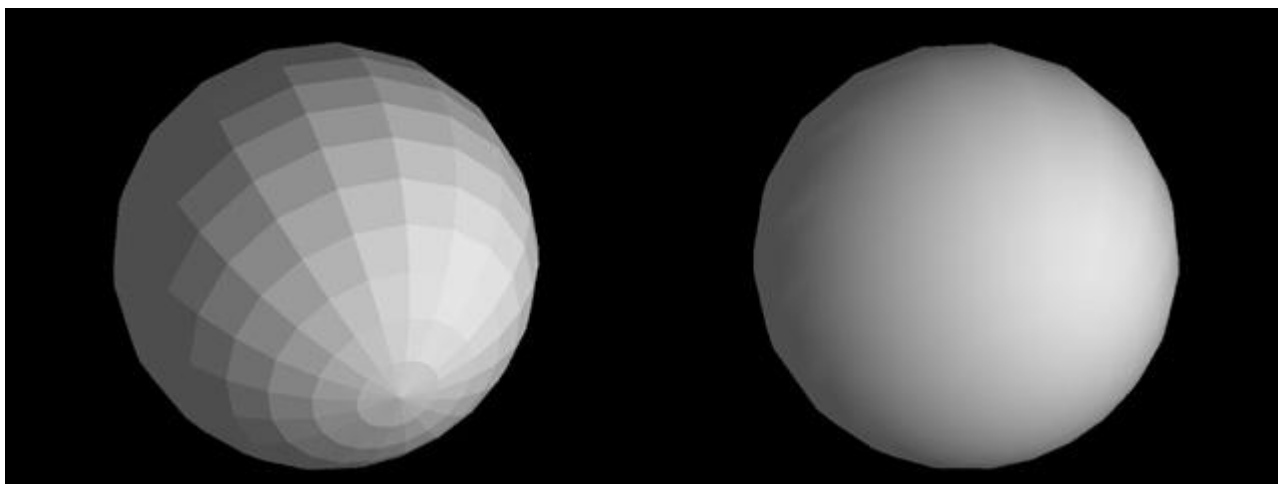
```

11. `Vertex_Normal[2] = Vbuffer[2] / 3;`

Важно е да се отбележи, че при изрисуването на обекта, за да посочат правилно върховете (вертексите) нормали на **OpenGL** трябва да се извика функцията **glNormal*()** всеки път, когато се определя връх с функцията **glVertex*()**. Ако някаква фигура е съставена от много триъгълници то за нейното реалистично оцветяване е нужно с помощта на цикъл да се изрисува всеки един от тях. Пример за използване на вертексни нормали :

```
1. glBegin( GL_TRIANGLES );
2. for( ... )
3. {
4.   glNormal3f( ... ); glVertex3f( ... );
5.   glNormal3f( ... ); glVertex3f( ... );
6.   glNormal3f( ... ); glVertex3f( ... );
7. }
8. glEnd( );
```

След определянето на нормалите на даден обект не трябва да се променя размера на обекта с функцията **glScale*()** защото няма да съответстват на нормалите. Пример за разликата между нормалите за лице и вертексите нормали :



Готовите обекти от **GLAUX** и **GLUT** имат предварително определени нормали за всеки връх. За да се използват нормални вектори за всяко лице, а не за всеки връх, просто трябва да се извикат вече познатата ви функция **glShadeModel(GLenum mode)** с един от аргументите :
GL_FLAT - за едноцветни полигони (понеже всеки полигон може да има само един цвят, ще получите "плочковидния" ефект на нормалите за лице)
GL_SMOOTH - за преливащи цветове (ефекта на вертексите нормали ще се запази)

12.4 OPENGL - ДИНАМИЧНА СВЕТЛИНА

Динамичната светлина е изключително важна за една тримерна графика, понеже тя кара компютърното изображение да изглежда реалистично. За да има ефект динамичната светлина, трябва да са дефинирани нормалите на обектите. Ако се използват функциите за изграждане на готови обекти с **GLUT** или **GLAUX** то те са установени, но в противен случай ще трябва да се определят нормалите на обектите. Изчисляването на нормали (нормални вектори) е обяснено в 11.3. Ако се използва някой обект, генериран от функция на **GLUT** или **GLAUX**, за да се включи динамична светлина, трябва да се изпълнят две стъпки: дефиниране на източника на светлината и определяне на материала на обектите. Той влияе на начина, по който обектите ще отразяват даден тип светлина, от което зависи и как ще изглеждат те. Преди да започне с определянето на динамичната светлина трябва да се реализира с **OpenGL** алгоритмът за премахване на скрити линии за да се проверят кои

повърхнини са видими. Това става като се подаде на функцията **glEnable()** аргумента **GL_DEPTH_TEST**. Освен това за да се използва динамичната светлина, тя трябва да бъде включена чрез извикване отново **glEnable()**, но с аргумента **GL_LIGHTING**. Тези две функции както и всички по-надолу е строго препоръчително да бъдат извикани предварително в **main / WinMain** (ако ползвате **Win32 Api**) след като вече сте направили прозореца за вашето приложение, а не във вашата рендерираща функция.

В **OpenGL** съществуват е предвидено определянето на *три вида светлина*:

Разсеяна - Ambient - тази светлина идва отовсякъде и се отразява навсякъде. Този вид светлина се получава когато светлината от даден източник се отрази многократно във всички обекти. Т.е. **Ambient** светлината е глобалната светлина, която ни позволява да виждаме обектите по нашата сцена. Аналогично в слънчево време всичко изглежда по-светло, а в облачно - по-тъмно. Можете да дефинирате глобална **Ambient** светлина.

Дифузна - Diffuse - тази светлина идва от определена посока и се отразява навсякъде, проникваща светлина.

Отразена светлина от светлинен източник- Specular - тази светлина идва от определена посока и се отразява в определена посока. Това е основната компонента на реалистичния цвят, отразеният лъч достигащ до очите на наблюдателя.

След като се включи динамичната светлина трябва да се зададе параметрите на светлинния източник. Това става с една от двете разновидности на следната функция :

glLightf(GLenum light, GLenum pname, GLfloat param)

glLightfv(GLenum light, GLenum pname, const GLfloat *params)

Тя приема три аргумента. Първият е името на светлинния източник, което се определя чрез една от константите **GL_LIGHT0, GL_LIGHT1, GL_LIGHT2, GL_LIGHT3, GL_LIGHT4, GL_LIGHT5, GL_LIGHT6** или **GL_LIGHT7**. В **OpenGL** може да има едновременно осем действащи светлинни източника. Вторият аргумент определя типа на светлината и може да бъде един от аргументите : **GL_DIFFUSE, GL_SPECULAR, GL_AMBIENT** - определя разсеяната светлина излъчването на източника.

Трябва да се използва функцията **glLightfv()**, като се подаде за трети аргумент **float** масив от стойности, определящи нейното излъчване. Може да се определи цвят на светлината, като се сложи в масива съответната **RGBA** стойност. Например:

float m[] = { 0.2 , 0.2 , 0.2, 1.0 }; (четвъртото число се отнася за прозрачността, оставете го единица)

Пример за да определяне **Ambient** излъчването за светлинен източник **GL_LIGHT0** :

glLightfv(GL_LIGHT0 , GL_AMBIENT , m);

Обикновено светлинните източници не излъчват разсеяна **Ambient** светлина, а се определя глобална разсеяна **Ambient** светлина, което е описано по-надолу.

GL_DIFFUSE - определя **Diffuse** излъчването на източника. Трябва да използвате **glLightfv()**, като се подаде съответния масив със стойности за цвят.

GL_SPECULAR - определя **Specular** излъчването на източника. Отново трябва да се използват **glLightfv()**, като се подадат съответния масив със стойности за цвят.

GL_POSITION - определя координатите на светлинния източник. Трябва да се използва **glLightfv()**, като се подаде съответния масив със стойности, определящ (x , y , z , (w)) позицията на източника. Ако имате w аргумент и той е равен на 0, това означава че източника се намира на безкрайно разстояние и съответните стойности определят посоката на светлината.

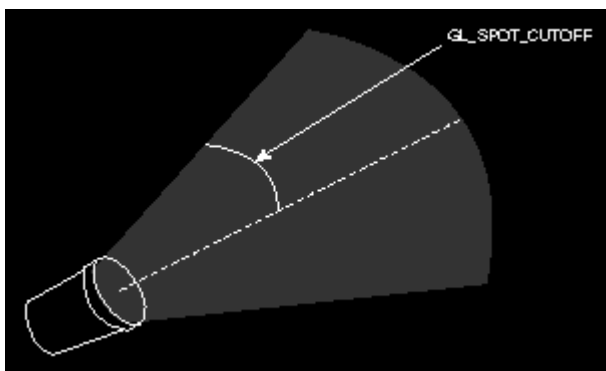
GL_SPOT_DIRECTION - определя посоката на разпространение на светлината. Трябва да използвате **glLightfv()** , като подадете съответния масив със стойности.

GL_SPOT_EXPONENT - определя фокуса (силата) на светлината. Той може да бъде стойност от 0 - 128. Колкото е по-голям, толкова светлината е по-ярка в центъра и по-слаба по краищата на конуса. Вече трябва да използвате **glLightf()** . Например:

```
glLightf ( GL_LIGHT0 ,GL_SPOT_EXPONENT ,10.0 );
```

GL_SPOT_CUTOFF - определя ъгъла между вектора сочещ посоката на светлината и края на конуса, в който се разпространява светлината. Може да приема стойности от 0 до 90. При по-големи от 90 градуса светлината се разпространява навсякъде и няма конус. Трябва да се използва **glLightf()** , като се подадат за трети аргумент съответния ъгъл. Например :

```
glLightf ( GL_LIGHT0 , GL_SPOT_CUTOFF , 45.0 );
```



За да се определи всичко необходимо за определен светлинен източник трябва да се извика последователно няколко пъти **glLightf()** / **glLightfv()** , като може да се дефинира няколко вида светлина, които се излъчват от един източник.

След това трябва да се включи съответния светлинен източник със функцията **glEnable()** , като подадете за параметъра неговото име. Например : **glEnable(GL_LIGHT0)** . Той може да бъде и изключен с функцията **glDisable()** .

Материала на обектите се определя по следния начин. Това става с една от разновидностите на следната функция :

```
glMaterialfv( GLenum face, GLenum pname, const GLfloat *params )
```

```
glMaterialf( GLenum face, GLenum pname, GLfloat param )
```

Тя приема три аргумента. Първия аргумент може да бъде една от константите **GL_FRONT** , **GL_BACK** или **GL_FRONT_AND_BACK** , които определят за коя част от обекта се отнася съответната дефиниция на материала. Вторият аргумент може да бъде един от аргументите:

GL_AMBIENT - определя начина на отразяване на светлина тип **Ambient**. Трябва да се използва **glMaterialfv()** , като се подаде съответния масив от стойности. Това може да бъде същия масив, който е използван при дефинирането на излъчваната светлина от определен източник.

GL_DIFFUSE - определя начина на отразяване на светлина тип **Diffuse**. Трябва да се използва **glMaterialfv()** , като се подадете съответния масив от стойности.

GL_SPECULAR - определя начина на отразяване на светлина тип **Specular**. Използва се **glMaterialfv()** , като се подаде съответния масив от стойности.

GL_EMISSION - определя излъчвана от обекта светлина. Тя се отразява единствено на цвета на обекта и може да бъде използвана ако искате да създадете лампа. Трябва да се използва **glMaterialfv()** , като се подаде съответния масив от стойности.

GL_SHININESS - определя големината на най-яркото отражение, причинено от светлина тип **SPECULAR**. С **glMateriali()** подава се число от 0 до 128. Колкото по-голямо е то, толкова по-точно фокусирано и по-малко е светлинното петно. Определянето на най-яркото

изображение при използването на светлина тип **SPECULAR** е много важно за реалистичното оцветяване.

GL_AMBIENT_AND_DIFFUSE - определя едновременно еднакъв начин на отразяване за **Ambient** и **Diffuse** светлината.

Ако обектите са били с предварително дефиниран цвят, то при включване на динамичната светлина тяхния цвят не се взема в предвид, а се генерира нов цвят в зависимост от светлинните източници и начина на отразяване на светлината. Ако обаче се подаде на **glEnable()** константата **GL_COLOR_MATERIAL** предварителният цвят на обектите взема участие в създаването на крайния цвят. Това не е препоръчително, ако светлината не е само бяла т.е. ако **RGB** стойностите в масива не са еднакви. Например: **float Diffuse_light[] = { 0.8 , 0.8 , 0.8 , 1.0 };**

Цвят на материала се задава с функцията :

glColorMaterial(GLenum face, GLenum mode) - тя определя за кои видове светлина да се взема в предвид цвета на обекта при изграждането на крайния му цвят. Първия аргумент може да бъде една от вече познатите ви константи **GL_FRONT**, **GL_BACK** или **GL_FRONT_AND_BACK**, а втория - **GL_EMISSION**, **GL_AMBIENT**, **GL_DIFFUSE**, **GL_SPECULAR** или **GL_AMBIENT_AND_DIFFUSE** т.е. съответно светлината при която ще влиза в сила и цвета на обекта.

Желателно е всеки път да определяте глобална **Ambient** светлина вместо да определяте за даден светлинен източник **Ambient** светлината, която излъчва, ако има такава. Това става с функцията :

glLightModelfv(GLenum pname, const GLfloat *params) - тя приема два аргумента. За се определи глобална **Ambient** светлина трябва да се подаде за първи аргумент константата **GL_LIGHT_MODEL_AMBIENT**, а за втори - масива, който определя излъчването на **Ambient** светлината. Определянето на глобална **Ambient** светлина може да бъде изключително полезно. Ако не се определи никаква **Ambient** светлина, това не значи че няма да има такава. По подразбиране **Ambient** светлината в **OpenGL** е **ambient_light[] = { 0.2 , 0.2 , 0.2 , 1.0 }.**